

 Lists & Slicing

INSTANT

 Tuples & Immutability Data Analysis Diploma • Session 11

PYTHON

DATA STRUCTURES

Lists · Tuples · Dictionaries · Sets · Operations & Comprehensions

4 Structures

Core Foundations

Lists • Tuples • Dicts • Sets

20 Slides

Curriculum Depth

Theory & Interactive Labs

Live Demos

Hands-on Practice

Indexing, Methods & Math

[Start Session 11 →](#) Dicts & O(1) Lookup Sets & Unique Ops

— WHAT WE WILL COVER

Master the 4 Python Data Structures

Understanding when and how to store, access, and manipulate data efficiently:

01



LISTS

Ordered, mutable collections. Create, access, slice, and use powerful list methods and comprehensions.

02



TUPLES

Ordered, immutable collections. Understand when and why to use unchangeable data.

03



DICTIONARIES

Key-value paired data. The most powerful structure for labeled, fast-lookup information.

04



SETS

Unique, unordered collections. Eliminate duplicates and perform mathematical set operations.



PART 01

LISTS

Ordered, Mutable Collections

Learn to create, access, modify, and manipulate ordered collections of data.

— UNDERSTANDING LISTS

What Are Lists & Why Use Them?

Definition: Lists are ordered, mutable collections of items that can contain different data types.

WHY

You need lists when you want to store multiple related items in a single variable, access them by position, and modify them over time.

Analogy: A shopping list — ordered, flexible, holds different things.

HOW

Create with square brackets [], access by index (starts at 0), and modify elements directly by assignment.

KEY CHARACTERISTICS

- Ordered
- Mutable
- Allows Duplicates
- Mixed Data Types

</> PYTHON EXAMPLE

Python 3.x

```
# Creating a list
fruits = ["apple", "banana", "cherry"]

# Access by index
print(fruits[0]) # "apple"
print(fruits[-1]) # "cherry"

# Modify an element
fruits[1] = "blueberry"

# Mixed data types
mixed = ["hello", 42, 3.14, True]

# Check length
```

INDEXING & SLICING

Creating & Accessing Lists

CREATING LISTS

```
# Empty list
empty = []

# List with values
numbers = [1, 2, 3, 4, 5]
mixed = ["hi", 10, 2.5]
```

ACCESSING & SLICING

- Index starts at 0 (not 1!)
- `my_list[0]` → first item
- `my_list[-1]` → last item
- Negative indices count from end (-1 last, -2 second-to-last).

</> INTERACTIVE INDEX VISUALIZATION

```
items = ["A", "B", "C", "D", "E"]
```

Forward Indices (0 to 4)

idx: 0	idx: 1	idx: 2	idx: 3	idx: 4
"A"	"B"	"C"	"D"	"E"
neg: -5	neg: -4	neg: -3	neg: -2	neg: -1

Select Slicing Preset:

items[1:4]

items[:3]

items[2:]

items[:]

Result: `print(items[:])` → ["A", "B", "C", "D", "E"]

— LIST METHODS

List Methods — Your Toolkit

ADDING ITEMS

- `.append(item)` — Add to end
- `.insert(idx, item)` — Add at position
- `.extend(list)` — Merge list

REMOVING ITEMS

- `.remove(item)` — Remove first match
- `.pop(idx)` — Remove & return
- `.clear()` — Empty list

FINDING & ORDERING

- `.index(item)` — Find position
- `.sort()` — Order items
- `len(list)` — Count items

⚡ LIVE METHOD DEMO EXPLORER

[Reset List](#)

```
tasks = [ "read", "code", "test" ]
```

```
len: 3
```

+ .append("deploy")

+ .insert(1, "plan")

🗑️ .remove("test")

🗑️ .pop()

↕️ .sort()

Initial List: ["read", "code", "test"]

— LIST COMPREHENSIONS

List Comprehensions — Pythonic Power

A concise way to create lists in a single line. More readable, faster, and considered "Pythonic".

SYNTAX PATTERNS

Basic:

```
[expression for item in iterable]
```

With filter:

```
[expression for item in iterable if condition]
```

Squares of 0-9

```
squares = [x**2 for x in range(10)]
```

```
→ [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Even numbers only

```
evens = [x for x in range(20) if x % 2 == 0]
```

```
→ [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

OLD WAY VS NEW WAY

LOOP WAY (4 lines):

```
squares = []
for x in range(10):
    squares.append(x**2)
```

COMPREHENSION (1 line):

```
squares = [x**2 for x in range(10)]
```



Pro Tip: List comprehensions are faster because they are optimized internally in C.

— REAL-WORLD EXAMPLES

Lists in Action — Real-World Examples

✓ To-Do List

```
tasks = ["Study", "Code", "Review"]
tasks.append("Sleep")
```

Track and manage daily tasks dynamically

|| Student Grades

```
grades = [85, 92, 78, 96]
avg = sum(grades) / len(grades)
```

Calculate average: 87.75

📦 Store Inventory

```
inventory = ["laptop", "mouse", "keyboard"]
if "mouse" in inventory: print("In stock!")
```

Check stock with 'in' operator

🔍 User Feedback

```
feedback = ["Great!", "Needs work", "Excellent"]
positive = [f for f in feedback if "Great" in f]
```

Filter responses matching criteria

Key Insight:

Lists are your go-to structure when you need an ordered collection that can change over time. They are the most versatile and commonly used data structure in Python.



PART 02

TUPLES

Ordered, Immutable Collections

When data should stay fixed and protected from accidental changes.

— UNDERSTANDING TUPLES

What Are Tuples & Why Use Them?

Definition: Tuples are ordered, immutable collections — once created, they cannot be changed.

WHY

Use tuples to protect data from accidental modification, or when you need a collection usable as a dictionary key.

Analogy: GPS coordinates (lat, long) — fixed together, shouldn't change.

HOW

Create with parentheses () or just commas. Access by index just like lists.

KEY CHARACTERISTICS

☒ Ordered
 ☒ Immutable
 ☒ Allows Duplicates
 ☒ Faster

</> PYTHON EXAMPLE

Python 3.x

```
# Creating tuples
coords = (40.7, -74.0)
rgb = (255, 128, 0)
single = ("only",) # comma required!

# Accessing (same as lists)
lat = coords[0] # 40.7
lng = coords[1] # -74.0

# Slicing works too
partial = rgb[:2] # (255, 128)

# This WILL FAIL:
# coords[0] = 41.0 # TypeError!
```

— UNPACKING & PATTERNS

Tuple Operations & Unpacking

TUPLE METHODS (ONLY 2!)

Because tuples are immutable, they only have 2 methods:

- `.count(item)` – counts occurrences
- `.index(item)` – finds position

PACKING & UNPACKING

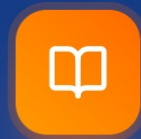
```
x, y = (10, 20) # x=10, y=20
first, *rest = (1, 2, 3, 4) # first=1, rest=[2,3,4]
a, b = b, a # Elegant swap!
```

WHEN TO USE TUPLES

- ✓ Data should never change (coordinates, dates)
- ✓ You need to use it as a dictionary key
- ✓ You want better performance than lists
- ✓ Returning multiple values from a function

NESTED TUPLES

```
matrix = ((1, 2), (3, 4), (5, 6))
print(matrix[0][1]) # 2
print(matrix[2]) # (5, 6)
```



PART 03

DICTIONARIES

Key-Value Paired Data

The most powerful structure for labeled, fast-lookup information in Python.

— UNDERSTANDING DICTIONARIES

What Are Dictionaries & Why Use Them?

Definition: Dictionaries store data as key-value pairs — look up a key to get its value instantly.

WHY

Use dictionaries when you need fast lookups by a meaningful identifier rather than by position number.

Analogy: A phone book — look up a name (key) to find a number (value).

HOW

Create with curly braces {}. Access values by key, not by index.

KEY CHARACTERISTICS

✓ Key-Value Pairs

✓ Mutable

✓ Unique Keys Only

</> PYTHON EXAMPLE

Python 3.x

```
# Creating a dictionary
user = {
    "name": "Alice",
    "age": 25,
    "city": "Cairo"
}

# Access by key
print(user["name"]) # "Alice"

# Safe access
print(user.get("job", "N/A")) # "N/A"

# Add/Update
```

— DICTIONARY METHODS & ITERATION

Dictionary Methods & Iteration

ADDING & REMOVING

- `dict["key"] = val`
- `.update(other)`
- `.pop("key")` / `.clear()`

ACCESSING SAFELY

- `.get("k", default)`
- `.keys()` / `.values()`
- `.items()`

CHECKING & LOOKUP

- `"key" in dict` ($O(1)$!)
- `len(dict)`
- Instant lookup speed

ITERATING OVER DICTIONARIES

```
user = {"name": "Alice", "age": 25, "city": "Cairo"}

# Iterate keys
for key in user:
    print(key, user[key]) # name Alice, age 25, city Cairo

# Iterate key-value pairs
for key, value in user.items():
    print(f"{key}: {value}") # name: Alice, age: 25, city: Cairo
```

— COMPREHENSIONS & REAL USES

Dictionary Comprehensions & Real Uses

SYNTAX

```
{key: value for item in iterable if condition}
```

QUICK EXAMPLES

```
{x: x**2 for x in range(5)} → {0:0, 1:1, 2:4, 3:9, 4:16}
{w: len(w) for w in ["hi", "hello"]} → {"hi":2, "hello":5}
```

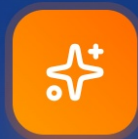
⚡ REAL CODE DEMO: WORD FREQUENCY

Type text to count word frequencies:

the cat and the hat

Frequency Dict Result:

```
{ "the": 2, "cat": 1, "and": 1, "hat": 1 }
```



PART 04

SETS

Unique, Unordered Collections

Eliminate duplicates and perform mathematical set operations with ease.

— UNDERSTANDING SETS

What Are Sets & Why Use Them?

Definition: Sets are unordered collections of unique items — no duplicates, no guaranteed order.

WHY

Use sets to eliminate duplicates, test membership, or perform mathematical operations (union, intersection).

Analogy: A set of unique student IDs — each has one ID, order doesn't matter.

HOW

Create with curly braces {} or set(). Note: {} creates an empty dict, use set() for empty set.

Important: Sets only store immutable items — you can't put a list inside a set!

</> PYTHON EXAMPLE

Python 3.x

```
# Creating sets
nums = {1, 2, 3, 3, 3}
print(nums) # {1, 2, 3} — dupes gone!

# From a list
unique = set([1, 2, 2, 3])
print(unique) # {1, 2, 3}

# Empty set (NOT {})
empty = set() # Correct!

# Membership test
print(2 in nums) # True — very fast!
```

— SET OPERATIONS

Set Operations — Math in Action

$A \cup B$ (.union())

All unique elements

$A \cap B$ (.intersection())

Common elements

$A - B$ (.difference())

In A but not B

$A \Delta B$ (.symmetric_diff())

In either, not both

$A = \{1, 2, 3, 4\}$ | $B = \{3, 4, 5, 6\}$

Interactive Set Math

Result Set: { 1, 2, 3, 4, 5, 6 }

❄️ **Frozen Set:** frozenset([1, 2, 3]) — immutable set usable as a dictionary key!

— SUMMARY & DECISION MATRIX

The Big Comparison — When to Use What

Feature	List []	Tuple ()	Dict { }	Set { }
Ordered	Yes	Yes	Yes*	No
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Keys: No	No
Access	By index	By index	By key	Membership

*Python 3.7+ maintains insertion order for dictionaries

Need ordered data?

Use a LIST

Data must not change?

Use a TUPLE

Need key-value pairs?

Use a DICT

Need uniqueness?

Use a SET

INSTANT

🕒 SESSION 11 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

Next Session: Python Libraries – NumPy, Pandas & Data Visualizations

🔄 Restart Presentation

Open Session 01: Excel Basics →